

从零开始移植Bootloader(极速版)

注意事项:

1. 本文章只涉及如何移植并简单使用!!!
2. 其中bootloder与ringbuffer知识请自行查询ai
3. 其中所有的工程代码请自行查询我的github/gitee上进行拷贝学习
4. 其中涉及路径最好不要包含中文

使用模块以及平台说明:STM32F407VET6(嘉立创的天空星)+HAL库

BL工程创建与使用

1. 文件夹创建

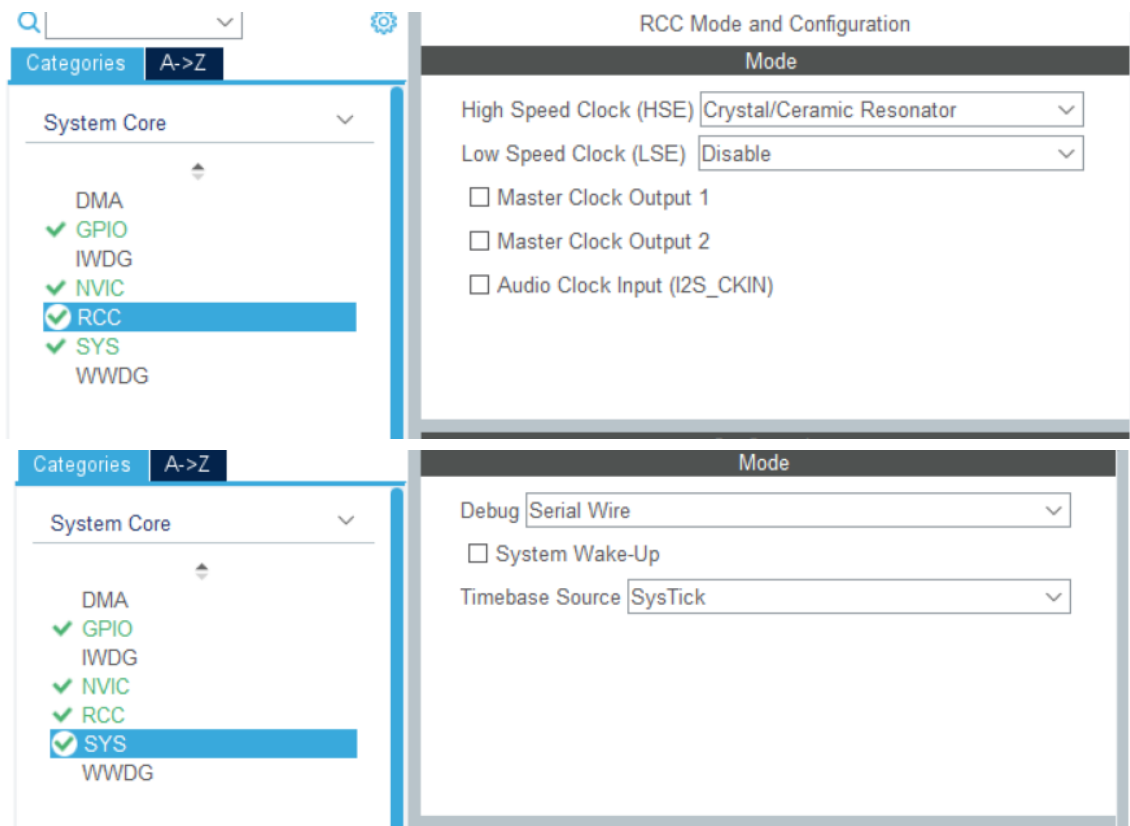
BL+BL_APP+APP+BL_min+Common(共5个文件夹)

2. 使用STM32CubeMX完成工程创建

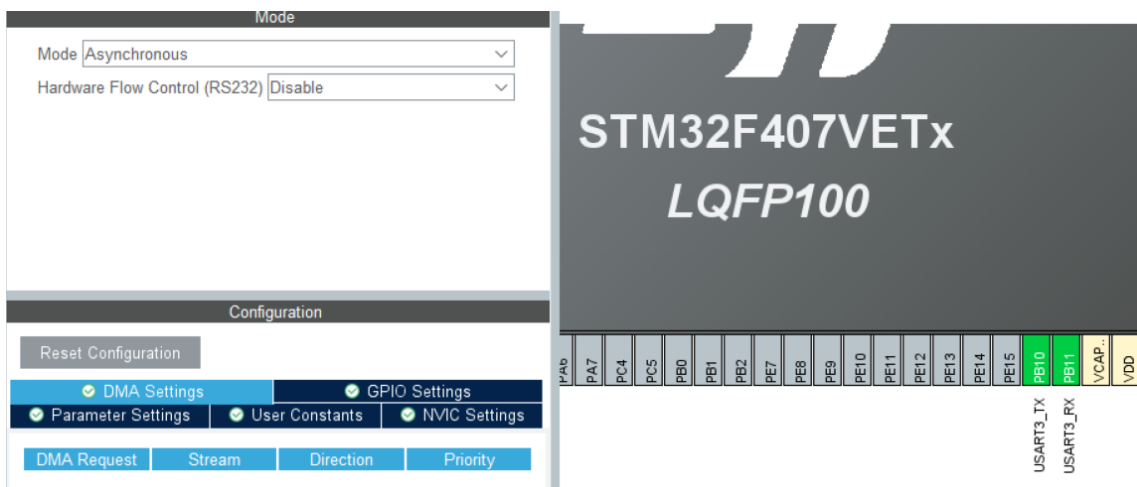
1. 选择芯片STM32F407VET6
2. 修改文件名以及选择工程存放路径等操作(这是我的设置, 仅供参考)

Pinout & Configuration	Clock Configuration	Project Manager	Tools
Project	Project Settings Project Name: BL Project Location: F:\西门子嵌入式+51\Bootloader\BL Browse Application Structure: Advanced Do not generate the main() Toolchain Folder Location: F:\西门子嵌入式+51\Bootloader\BL\BL\ Toolchain / IDE: MDK-ARM Min Version: V5.32 Generate Under Root		
Code Generator	STM32Cube MCU packages and embedded software packs ☒ Copy all used libraries into the project folder ☐ Copy only the necessary library files ☐ Add necessary library files as reference in the toolchain project configuration file Generated files ☒ Generate peripheral initialization as a pair of '.c/.h' files per peripheral ☐ Backup previously generated files when re-generating ☒ Keep User Code when re-generating ☒ Delete previously generated files when not re-generated		
Advanced Settings			

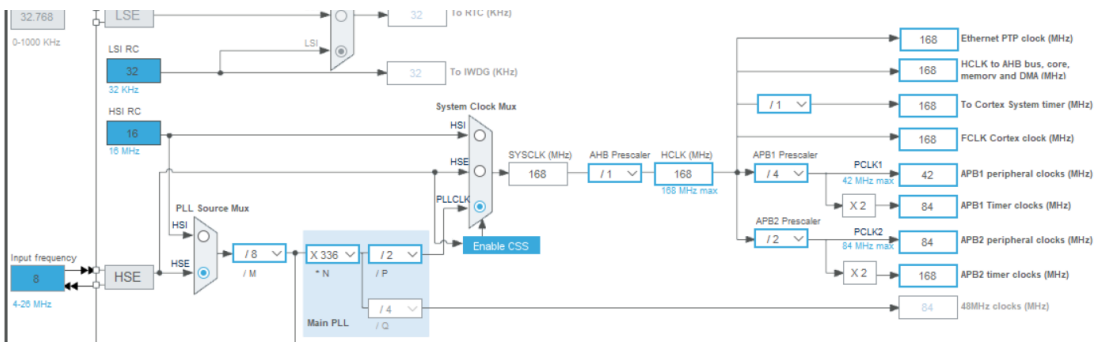
3. 将时钟与SYS一同打开



4. 打开串口3，并设置频率为500000 Bit/s(串口引脚可自行选择)-该串口3就是我们的OTA串口



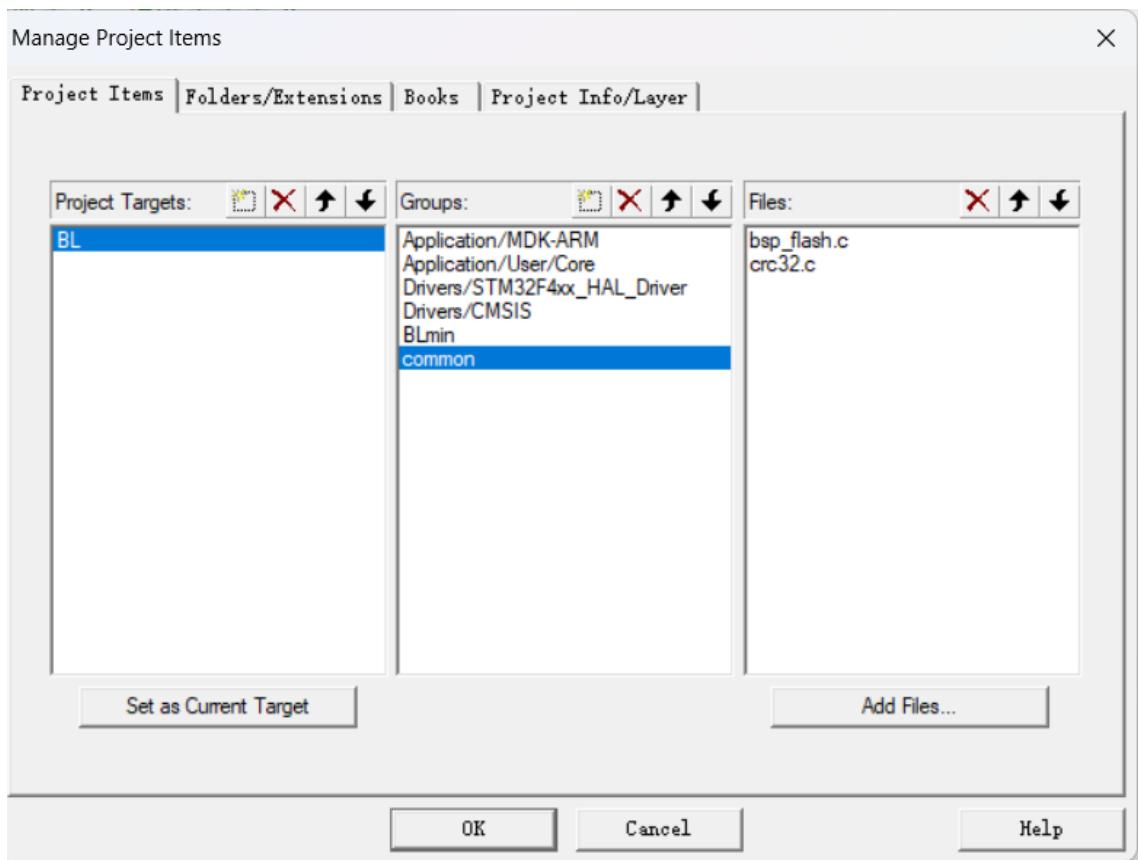
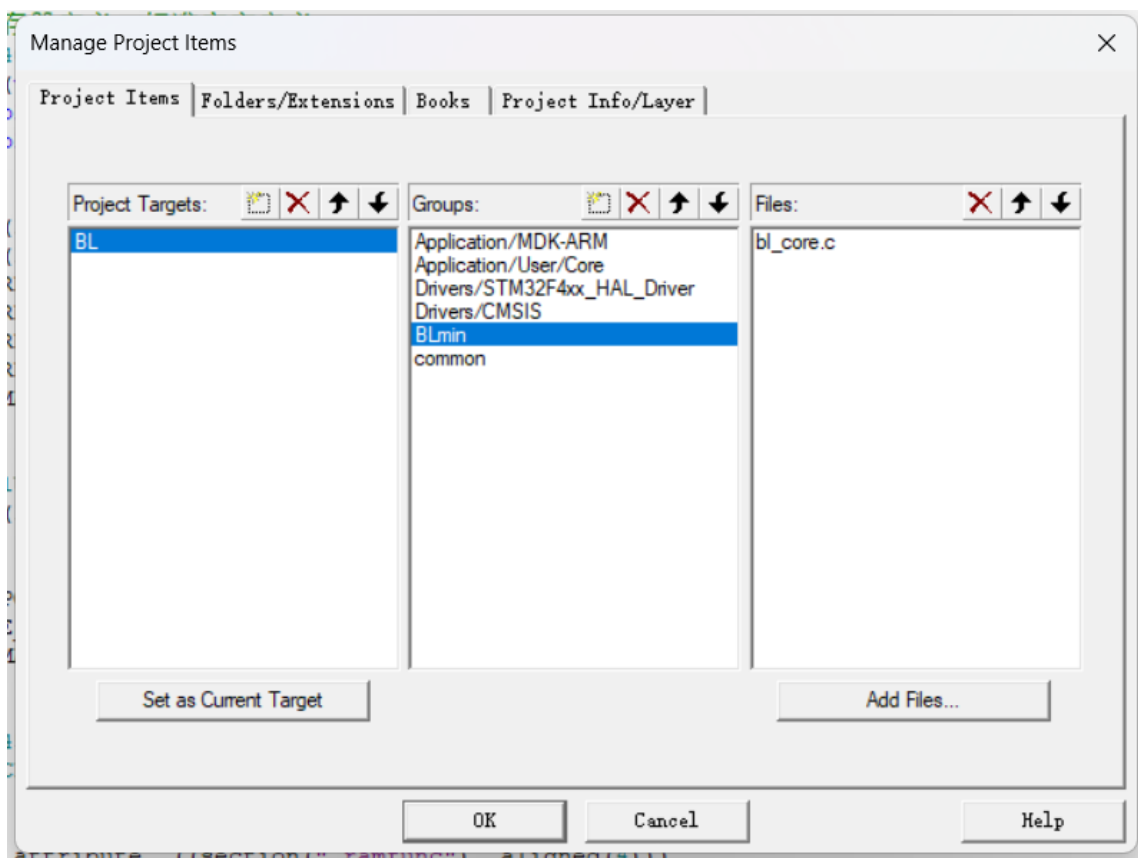
5. 设置时钟树(重点)



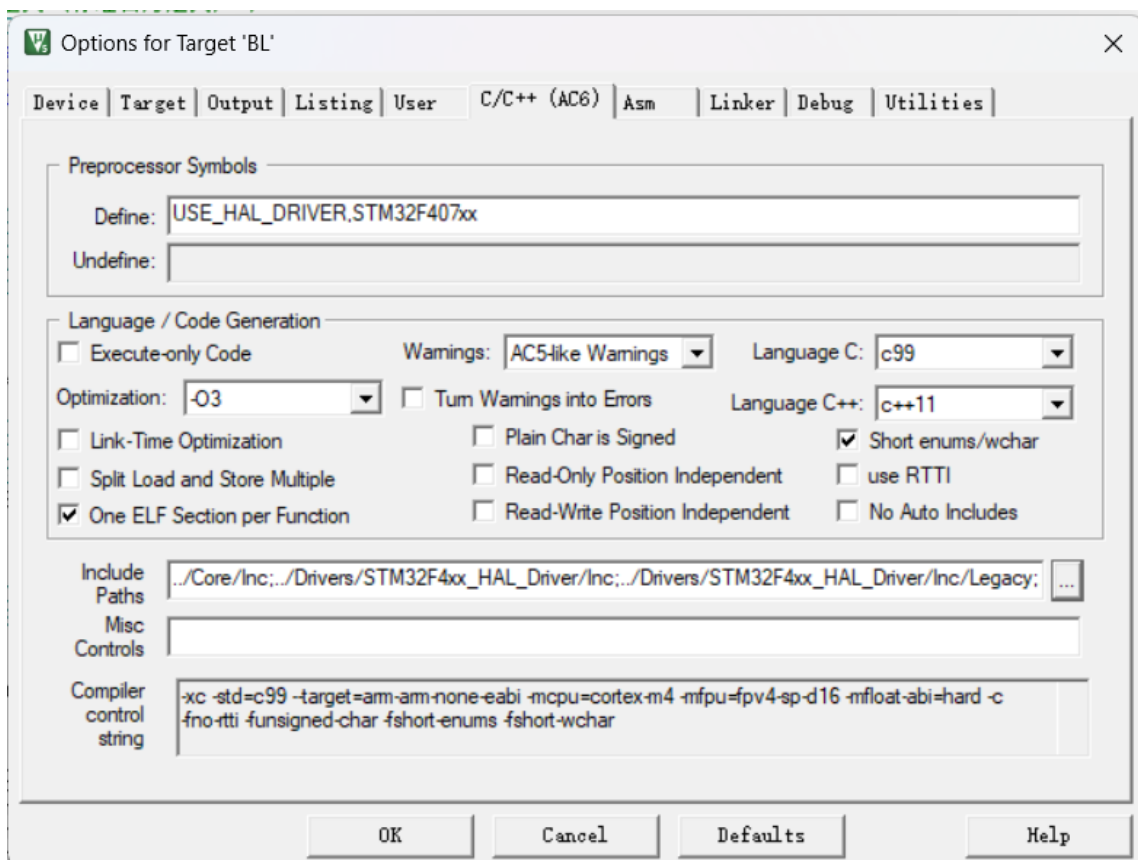
6. 生成工程代码

3.keil中文件的添加以及文件路径设置与代码添加

1. 添加文件如图所示



2. 点击魔法棒，添加头文件的路径(参考)-点击上面的Include Paths->然后添加刚刚创建好的BL_min和Common文件的路径即可(注意:一定要点进文件夹)

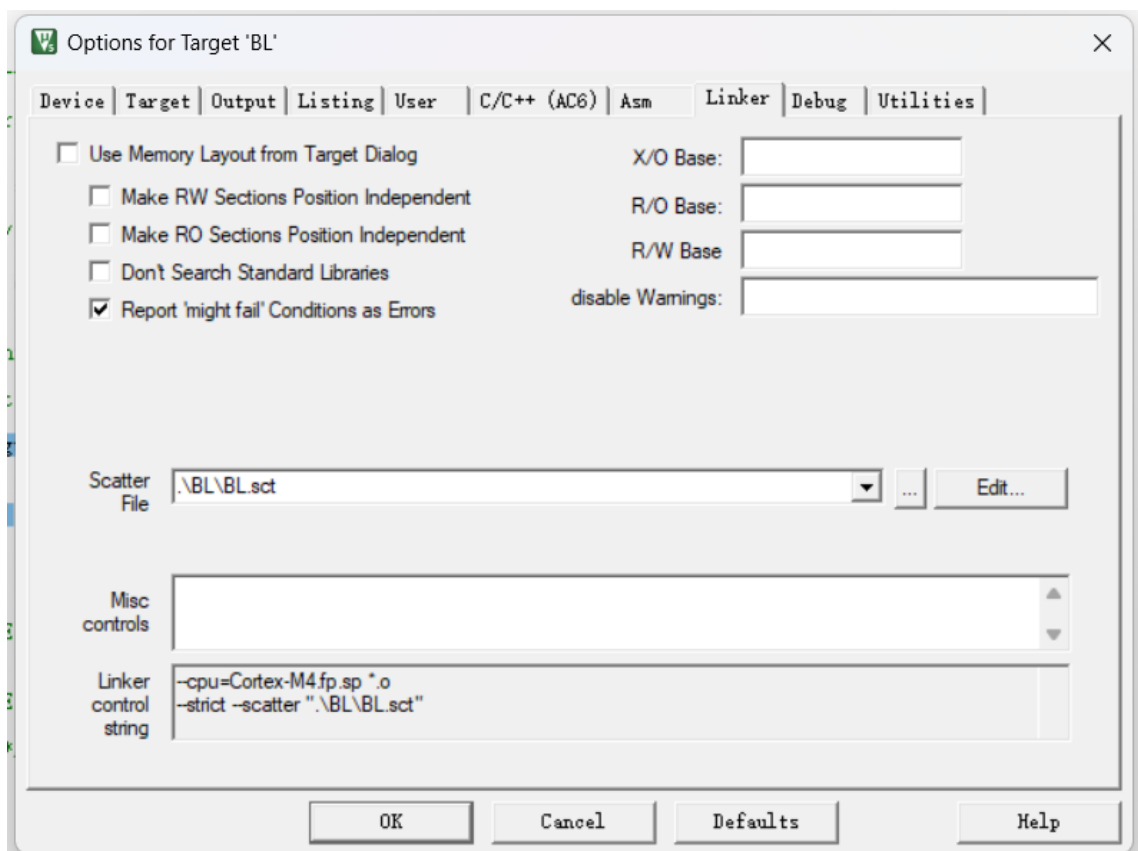


3. 打开main.c添加代码即可

```
/* USER CODE BEGIN Includes */
#include "bl_core.h"           //请添加在头文件引用中
/* USER CODE END Includes */
```

```
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_USART3_UART_Init();
/* USER CODE BEGIN 2 */
    bootloader_run();          //放置在硬件初始化后面
/* USER CODE END 2 */
```

4. 首先将Linker中取消勾选(如图所示)->修改MDK_ARM文件夹下的BL.sct代码(修改或者直接复制我代码中BL.sct)->在Scatter File中选择修改好的BL.sct路径(重点)



BL.sct代码

```
; *****
; *** Scatter-Loading Description File generated by uVision ***
; *****

LR_IROM1 0x08000000 0x0000C000 {      ; 48KB BL 空间（对齐 16KB，正确）
  ER_IROM1 0x08000000 0x0000C000 {
    *.o (RESET, +First)
    *(InRoot$$Sections)
    .ANY (+RO)
    .ANY (+XO)
  }
RW_IRAM1 0x20000000 0x00030000 { .ANY (+RW +ZI) } ; STM32F407 192KB RAM
ER_IRAM_FUNC +0 {
  bsp_flash.o (+RO)                      ; Flash 代码放 RAM 执行（正确）
}
}
```

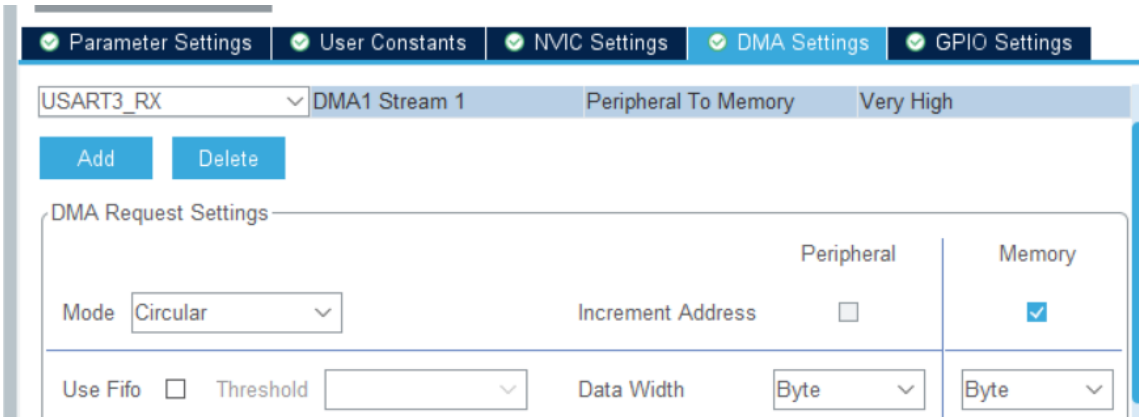
5. 编译下载->打开串口验证是否程序成功下载并运行(如图所示)

```
BL: --- log dump ---
BL: no valid APP, halt.
```

最小APP测试工程创建与使用

1.使用STM32CubeMX完成工程创建

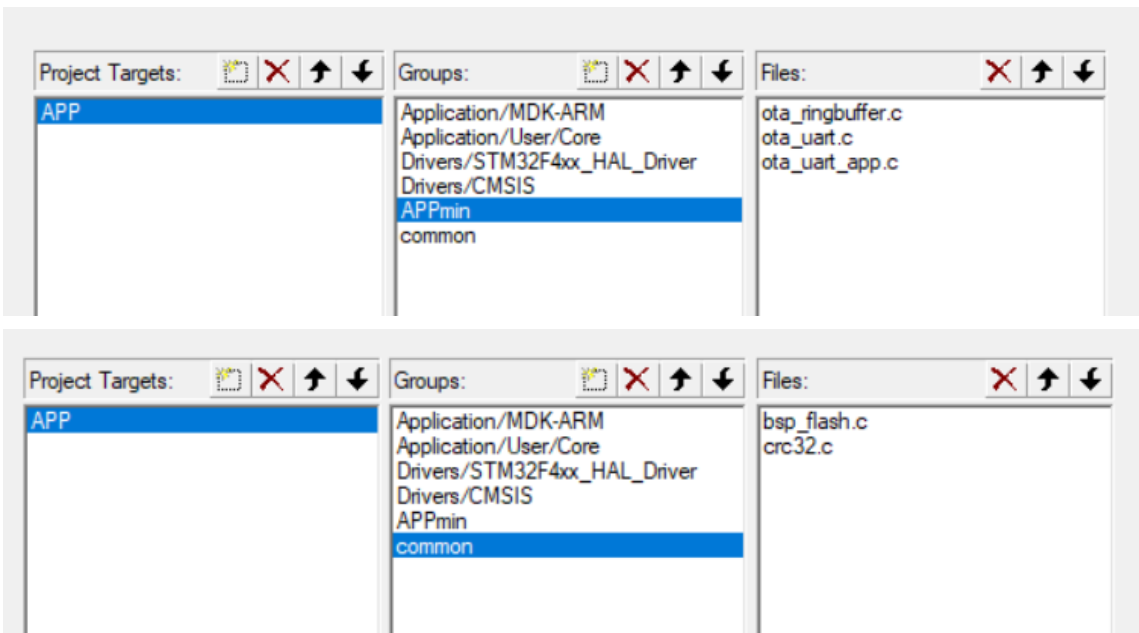
- 1. 创建项目流程与上方BL建立工程相同
- 2. 打开串口1(检验固件更新)-引脚自行选择
- 3. 选中串口3，对串口3的读取添加DMA(DMA设置如图所示)



- 4. 创建工程

2.keil中文件的添加以及文件路径设置与代码添加

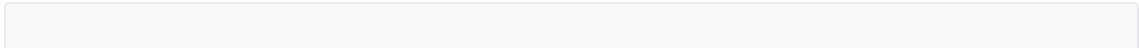
- 1. 添加文件如图所示



- 2. 点击魔法棒，添加头文件的路径(参考)-点击上面的Include Paths->然后添加刚刚创建好的BL_APP和Common文件的路径即可(注意:一定要点进文件夹)-操作与上方BL创建相同
- 3. 点开main.c补充代码

```
/* USER CODE BEGIN Includes */
#include "ota_uart_app.h"           //添加头文件定义
#define APP_BANNER  "APP v1.0 running\r\n"    // OTA 验证：下发前改成 "APP v2.0"
/* USER CODE END Includes */
```

主函数书写



```

int main(void)
{

    /* USER CODE BEGIN 1 */
    SCB->VTOR = 0x08014000UL;          //重点：防止中断跑飞
    /* USER CODE END 1 */

    /* MCU Configuration-----
    -*/

    /* Reset of all peripherals, Initializes the Flash interface and the
    SysTick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_USART1_UART_Init();
    MX_USART3_UART_Init();
    /* USER CODE BEGIN 2 */
    ota_uart_start();
    HAL_UART_Transmit(&huart1, (uint8_t *)APP_BANNER, sizeof(APP_BANNER) - 1,
100);
    uint32_t t_ota = HAL_GetTick(), t_led = t_ota, t_beat = t_ota;
    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
        uint32_t now = HAL_GetTick();
        if (now - t_ota >= 5) { t_ota = now; ota_poll_task(); }
        if (now - t_beat >= 1000) { t_beat = now;
            HAL_UART_Transmit(&huart1, (uint8_t *)APP_BANNER, sizeof(APP_BANNER) - 1,
100); }
        }
    /* USER CODE END 3 */
}

```

4. 首先将Linker中取消勾选(如上方BL的图所示)->修改MDK_ARM文件夹下的APP.sct代码(修改或者直接复制我代码中APP.sct)->在Scatter File中选择修改好的APP.sct路径(重点)

APP.sct代码

```

LR_IROM1 0x08014000 0x00068000 {
  ER_IROM1 0x08014000 0x00068000 {
    *.o (RESET, +First)
    *(InRoot$$Sections)
    .ANY (+RO)
    .ANY (+XO)
  }
  RW_IRAM1 0x20000000 0x00020000 { .ANY (+RW +ZI) }
  ER_IRAM_FUNC +0 {
    bsp_flash.o (+RO)
  }
}

```

5. 编译下载->打开两个串口验证是否程序成功下载并运行(如图所示)

OTA串口打印

BL: jumping to app...

测试串口打印

```

[22:21:31.910]收←◆APP v1.0 running
[22:21:32.917]收←◆APP v1.0 running
[22:21:33.913]收←◆APP v1.0 running

```

上位机下发固件包以及单片机工程的远程更新

1.修改int main()主函数

1. 将APP v1.0 running->APP v2.0 running->重新编译(不要进行下载)
2. 将MDK_ARM文件夹里面的APP.hex复制粘贴出来，放置到随便的路径位置(你知道把文件放置到哪个位置就行)->将这个APP.hex改名为v2.hex(这一步可做可不做)

2.打开Tools文件夹运行ota_gui.py即可(注意下载需要的库)

1. 选择对应的OTA的串口，新固件的位置，测试串口，波特率大小(如图所示)



2. 点击开始升级->查看新固件是否成功下载并打印对应的输出(如图所示)


```
进度
11548/11548 字节 100% 0.0 KB/s ETA 0s
DATA:46 ACK:48 NACK:0 重传:0 超时:0

日志 / 监测
22:36:14 --- 监听 APP 桐帽口 COM6@115200 ---
22:36:14 镜像 v2.hex size=11548 crc32=0x97DECAD2 chunk=256 hdr=v2
22:36:14 OTA口已打开 COM8@500000 (DTR/RTS off, write_timeout=4s)
22:36:14 发送自检 OK (0.1 ms)
22:36:14 == START (含 Flash 擦除, 等待设备 ACK) ==
22:36:17 [dev] 已确认, 开始发送 DATA...
22:36:17 [dev] <◆◆Z◆
22:36:17 [APP] APP v1.0 running
22:36:17 == END (length=0, crc32 字段携带整镜像 CRC) ==
22:36:18 END 已确认, 传输耗时 3.4s (均速 3.3 KB/s)。设备将复位进入 BootLoader 执行升级。
22:36:18 --- 监测 BootLoader 升级流程 20s ---
22:36:18 [dev] BL: APP2 ok, copying...
22:36:19 [dev] BL: update OK
22:36:20 [dev] BL: jumping to app...
22:36:20 [APP] APP v2.0 running
22:36:21 [APP] APP v2.0 running
22:36:22 [APP] APP v2.0 running
22:36:23 [APP] APP v2.0 running
22:36:24 [APP] APP v2.0 running
22:36:25 [APP] APP v2.0 running
22:36:25 用户请求停止-
22:36:25 OTA 流程结束。
22:36:25 OTA口已关闭。
```

3.注意事项:烧录固件时推荐使用USB-TTL, 否则部分固件有可能更新失败或者请求超时

恭喜您完成了bootloader的快速移植与使用, 感谢您的学习